Solving a Three-Gate Binary with GDB

A discovery-first reverse-engineering lab for beginners

Starting information: one Linux binary and the hint “cafe beef dude”

Learning goals

- Perform initial triage on an unfamiliar Linux executable.
- Explain ELF, x86-64, dynamic linking, PIE, ASLR, and stripping.
- Use strings, readelf, objdump, and GDB as complementary tools.
- Recognize comparison-and-branch gates in assembly.
- Calculate runtime addresses in a PIE executable.
- Alter control flow with GDB and explain why the changes work.

Lab mindset: Do not begin by guessing the flag or jumping directly to an address. At each stage, ask: What evidence do I have? What hypothesis does it support? What should I test next?

1. Begin with the unknown file

Assume the analyst receives only a file named newestChallenge3. Before running it, confirm that it exists, inspect its permissions, and compute a hash so the exact sample can be identified later.

```
ls -l newestChallenge3
sha256sum newestChallenge3
```

For the supplied sample, the SHA-256 value is:

```
8a558112f0aaf240e2dd0639ed701c36fce1a1ac7f646ecb513a7672cb0de33b
```

Why hash first: If a classmate has a different hash, their offsets may differ even when the filename is identical.

2. Identify the file type

```
file newestChallenge3
```

The important output is:

```
ELF 64-bit LSB pie executable, x86-64, dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, ... stripped
```

2.1 What is ELF?

ELF stands for Executable and Linkable Format. It is the standard format used for executables, shared libraries, object files, and core dumps on most Linux systems. The ELF header tells tools how the file is organized and how it should be loaded.

2.2 What does 64-bit LSB x86-64 mean?

“64-bit” and “x86-64” identify the processor architecture and register width. “LSB” means least-significant byte first, another name for little-endian byte order. This determines which GDB register names and calling convention are relevant.

2.3 What does dynamically linked mean?

The executable relies on shared libraries loaded at runtime rather than containing every library routine itself. The interpreter path names the dynamic loader. Imported functions such as putc remain visible even when the program has been stripped because the loader must resolve them.

2.4 How do we know it is stripped?

The file command explicitly reports “stripped.” Verify this rather than relying on one tool:

```
nm newestChallenge3
readelf -s newestChallenge3 | less
```

Here, readelf -s displays the ELF symbol tables. The lowercase -s option means symbols. In an unstripped binary, this output often includes names for functions and variables. In this sample, most developer-defined names are absent, while a smaller set of imported dynamic symbols remains.

A stripped executable has had most normal symbol and debugging information removed. Names such as main and developer-defined helper functions are absent. The dynamic symbol table may still list imported library functions because dynamic linking requires them.

Practical consequence: Commands such as break main may fail. The analyst must work from the entry point, imported functions, instruction patterns, and offsets instead of friendly function names.

3. Determine whether the executable is PIE

The file command calls it a “pie executable,” but read the ELF header to understand why:

```
readelf -h newestChallenge3 | grep -E 'Type:|Machine:|Entry point'
```

The readelf -h option means header. It prints the main ELF header, including the file type, target architecture, byte order, entry-point address, and the locations of the program-header and section-header tables. The grep command only filters that larger output to the three fields most relevant at this stage.

```
Type: DYN (Position-Independent Executable file)
Machine: Advanced Micro Devices X86-64
Entry point address: 0x10a0
```

3.1 What does PIE mean?

PIE means Position-Independent Executable. Its machine code is built so the operating system can load the executable at different virtual addresses. In the ELF header, DYN stands for dynamic. The formal ELF type name is ET_DYN, traditionally used for shared objects that can be loaded at varying addresses. Modern PIE executables also use ET_DYN because they require the same relocation-friendly loading behavior. The presence of an entry point and program interpreter helps identify this ET_DYN file as an executable rather than an ordinary shared library.

3.2 Why does PIE matter in GDB?

Static disassembly displays offsets such as 0x140a. At runtime, the loader adds a randomized base address. Therefore, the actual instruction address is:

```
runtime address = PIE load base + static offset
```

A command such as break *0x140a usually fails because virtual address 0x140a is not mapped. The analyst must first determine the load base or disable randomization for the debugging session.

3.3 PIE and ASLR are related but different

PIE is a property of how the executable was built. ASLR is an operating-system behavior that randomizes memory locations when a process runs. PIE allows the executable itself to participate in ASLR. Disabling randomization in GDB makes addresses repeatable, but it does not turn the binary into a non-PIE executable.

4. Observe normal behavior

After making the file executable, run it without a debugger. Baseline behavior gives us something to compare against later.

```
chmod +x newestChallenge3
./newestChallenge3

Flag is: n0t_s0_fa5t
Have you even reversed it?
```

This output is suspicious for a reverse-engineering challenge. It openly mocks the analyst and may be a decoy. That is a hypothesis, not yet a fact. The next task is to determine whether more output or hidden data exists.

5. Search printable strings

```
strings -a -n 4 newestChallenge3 | less
```

The strings output is unusually sparse. The visible decoy text is not plainly stored as a complete string. This suggests that output may be generated character by character, decoded at runtime, or assembled from data that strings cannot recognize.

What strings cannot prove: Failure to find a flag does not mean a flag is absent. strings only reports contiguous printable byte sequences that satisfy its length threshold.

6. Inspect the ELF structure

```
readelf -S newestChallenge3
readelf -l newestChallenge3
```

6.1 What do the readelf options mean?

readelf examines the internal structures of an ELF file without executing it. The option determines which structure is displayed:

- -h or --file-header: display the main ELF header. Use it to identify the ELF class, endianness, machine architecture, file type, and entry point.
- -s or --symbols: display entries from symbol tables such as .symtab and .dynsym. This helps determine whether useful function and variable names remain.
- -S or --section-headers: display the section-header table. The uppercase -S is different from lowercase -s. It lists logical file sections such as .text, .rodata, .data, .bss, .dynsym, and .strtab.
- -l or --program-headers: display the program-header table, also called the segment table. It shows how the loader maps parts of the file into memory and which segments are readable, writable, or executable.

Sections primarily organize the file for linking and analysis. Segments describe the runtime memory image used by the operating-system loader. Several sections may be grouped into one loadable segment.

6.2 Interpreting the important structures

The section table shows .text for executable instructions, .rodata for read-only data, .data for initialized writable data, and .bss for zero-initialized data. The .dynsym section is the dynamic symbol table required for runtime linking; “dyn” in names such as .dynsym and .dynamic is an abbreviation for dynamic. The program headers show how these regions are mapped into memory. At this stage, no single section exposes the answer, so instruction-level analysis is appropriate.

7. Disassemble the program

Use Intel syntax because its operand order is often easier for beginners to read: destination first, then source.

```
objdump -d -Mintel newestChallenge3 > disassembly.txt
less disassembly.txt
```

Because symbols are stripped, objdump labels broad areas such as .text but cannot identify main by name. Begin near the ELF entry point, 0x10a0, and follow the startup code. Linux executables commonly pass the address of main to __libc_start_main.

```
objdump -d -Mintel --start-address=0x1080 --stop-address=0x1520 newestChallenge3
```

8. Use the hint as a search hypothesis

The hint contains three words: cafe, beef, and dude. In binary analysis, words composed of hexadecimal-looking letters often represent constants. CAFE and BEEF are valid hexadecimal values. “DUDE” is not valid because U is not a hexadecimal digit, but leetspeak commonly substitutes zeroes: D00D.

```
grep -Ein 'cafe|beef|d00d' disassembly.txt
```

The search reveals three immediate comparisons:

```
0x140a: cmp eax,0xcafe
0x14aa: cmp ecx,0xbeef
0x14ea: cmp eax,0xd00d
```

Reasoning checkpoint: The hint is now supported by direct evidence. The constants occur in executable code and are used in comparisons, so they are likely control-flow gates rather than decorative data.

9. Read each gate in context

A comparison is meaningful only when interpreted with the conditional jump that follows it. Display several instructions before and after each match.

```
objdump -d -Mintel --start-address=0x13f0 --stop-address=0x1510 newestChallenge3
```

Gate	Comparison	Following branch	Interpretation
1	0x140a: cmp eax,0xcafe	0x140f: je 0x1470	Continue to gate 2 only when equal.
2	0x14aa: cmp ecx,0xbeef	0x14b0: jne 0x1411	Return to decoy path when not equal.
3	0x14ea: cmp eax,0xd00d	0x14ef: jne 0x1411	Return to decoy path when not equal.

9.1 CMP, JE, and JNE

CMP performs a subtraction for the purpose of setting CPU status flags but does not save the subtraction result. JE, “jump if equal,” branches when the zero flag is set. JNE, “jump if not equal,” branches when the zero flag is clear.

The later two branches converge on offset 0x1411, which prints the decoy and exits. This shared destination is strong evidence that failing either gate returns the analyst to the false result.

10. Form a dynamic-analysis plan

Static analysis has produced a testable plan: pause at each comparison, inspect the compared register, and force execution onto the success path. There are two educationally useful methods:

- Modify the compared register so the original branch condition becomes true.
- Change RIP so execution resumes directly at the success continuation.

This walkthrough uses RIP redirection first because the challenge description emphasizes setting the execution address or skipping instructions. A later section demonstrates register modification.

11. Enter GDB and locate the PIE base

```
gdb -q ./newestChallenge3
(gdb) set disassembly-flavor intel
(gdb) set pagination off
```

Since main is stripped, break at the imported runtime function __libc_start_main. On x86-64 Linux, the first function argument is passed in RDI. At the call to __libc_start_main, RDI contains the runtime address of main.

```
(gdb) break __libc_start_main
(gdb) run
```

Static disassembly shows that main begins at offset 0x1080. Subtract that offset from the runtime main address to obtain the PIE base:

```
(gdb) p/x $rdi
(gdb) set $base = $rdi - 0x1080
(gdb) printf "PIE base = 0x%lx\n", $base
```

Verification: Use x/5i $base+0x1080. The instructions should match the static disassembly at offset 0x1080. This confirms both the base calculation and the assumed main offset.

12. Set runtime breakpoints at the gates

```
(gdb) break *($base + 0x140a)
(gdb) break *($base + 0x14aa)
(gdb) break *($base + 0x14ea)
(gdb) continue
```

13. Gate 1: redirect CAFE to its success block

When breakpoint 1 triggers, inspect the current instruction and nearby branch:

```
(gdb) x/4i $pc
(gdb) p/x $eax
```

Normal execution will not satisfy the intended comparison. Set RIP to the destination of the JE instruction, offset 0x1470:

```
(gdb) set $rip = $base + 0x1470
(gdb) x/i $pc
(gdb) continue
```

14. Gate 2: skip the BEEF failure branch

At gate 2, a failed comparison executes JNE and returns to the decoy block. Resume at the first instruction after the JNE, offset 0x14b6:

```
(gdb) x/4i $pc
(gdb) p/x $ecx
(gdb) set $rip = $base + 0x14b6
(gdb) continue
```

15. Gate 3: skip the D00D failure branch

Apply the same reasoning to the final gate. Resume after its JNE at offset 0x14f5:

```
(gdb) x/4i $pc
(gdb) p/x $eax
(gdb) set $rip = $base + 0x14f5
(gdb) continue
```

16. Confirm the result

```
Flag is: n0t_s0_fa5t
Congratulations! The correct flag is: r3v3rs1ng_1s_c00l!
```

Recovered flag: `r3v3rs1ng_1s_c00l!`

The decoy prefix still appears because the program begins emitting it before all three gates are bypassed. The final success message proves that execution reached code unavailable through the normal path.

17. Alternative solution: modify registers

Instead of changing RIP, set each compared register to the expected value immediately before CMP, then single-step through CMP and the branch. This lets the program take its own success branches and directly demonstrates flag behavior.

```
# Gate 1
set $eax = 0xcafe
nexti
nexti

# Gate 2
set $ecx = 0xbeef
nexti
nexti

# Gate 3
set $eax = 0xd00d
nexti
nexti
```

Use info registers eflags before and after CMP to observe the zero flag changing.

18. Automate only after understanding

Once the manual analysis is understood, save these commands as solve.gdb:

```
set pagination off
set confirm off
set disassembly-flavor intel
break __libc_start_main
run
set $base = $rdi - 0x1080
printf "PIE base: 0x%lx\n", $base

break *($base + 0x140a)
commands
  silent
  set $rip = $base + 0x1470
  continue
end

break *($base + 0x14aa)
commands
  silent
  set $rip = $base + 0x14b6
  continue
end

break *($base + 0x14ea)
commands
  silent
  set $rip = $base + 0x14f5
  continue
end

continue
```

```
gdb -q -x solve.gdb ./newestChallenge3
```

19. Troubleshooting

Symptom	Likely cause and correction
break main fails	The binary is stripped. Break at __libc_start_main or use the entry point.
Cannot insert breakpoint at 0x140a	0x140a is a PIE offset, not a mapped runtime address. Add $base.
Addresses change on a new run	ASLR selected a different base. Recalculate $base each run.
Only the decoy prints	A gate was not bypassed, or RIP was sent to the wrong continuation offset.
GDB reports an unknown architecture or file	Confirm the file hash and run file newestChallenge3.
The branch behaves unexpectedly	Stop on CMP, inspect the register, and single-step while checking EFLAGS.

20. Review questions

- Which part of file output proves that the program is PIE?
- Why can readelf report a dynamic symbol table even though the binary is stripped?
- Why is 0x140a not normally a valid runtime breakpoint by itself?
- What is the difference between changing RIP and changing EAX or ECX?
- Why does a JNE to 0x1411 identify 0x1411 as a failure path?
- How did the hint become evidence rather than a guess?

21. Answer key

Use this page after completing the lab questions.

1. Which part of file output proves that the program is PIE?

The phrase “pie executable” states it directly. readelf -h independently confirms it by reporting Type: DYN and identifying the file as a Position-Independent Executable.

2. Why can readelf report a dynamic symbol table even though the binary is stripped?

Stripping usually removes the full static symbol table and debugging symbols, but dynamically linked programs must retain the imported and exported symbols needed by the runtime linker. Those entries remain in .dynsym.

3. Why is 0x140a not normally a valid runtime breakpoint by itself?

It is a static offset inside a PIE file. At runtime, ASLR loads the executable at a base address, so the actual address is PIE base + 0x140a.

4. What is the difference between changing RIP and changing EAX or ECX?

Changing RIP bypasses instructions by redirecting control flow directly to a chosen address. Changing EAX or ECX preserves the original CMP and branch instructions but alters their input so the condition evaluates as successful.

5. Why does a JNE to 0x1411 identify 0x1411 as a failure path?

Both later gates jump there when their required values are not present, and execution at that location produces the decoy result and exits. Repeated convergence on the same destination is evidence that it is a shared failure block.

6. How did the hint become evidence rather than a guess?

The words were converted into plausible hexadecimal constants - CAFE, BEEF, and D00D - and those exact constants were then found in executable CMP instructions. Their placement in control-flow checks connected the hint to observable program behavior.

Command summary

```
file newestChallenge3
sha256sum newestChallenge3
nm newestChallenge3
readelf -h newestChallenge3
readelf -s newestChallenge3
readelf -S newestChallenge3
strings -a -n 4 newestChallenge3
objdump -d -Mintel newestChallenge3 > disassembly.txt
grep -Ein 'cafe|beef|d00d' disassembly.txt
gdb -q ./newestChallenge3
```